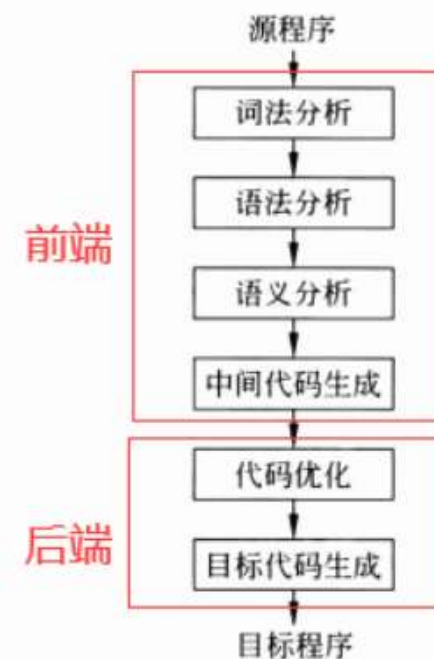


# 上下文无关语言

- 我们现在将从正则语言 转移到范围更大的一类语言，称为“上下文无关语言”。We now turn our attention away from the RL to a larger class of languages called the "context-free languages".
- 从高级程序语言的角度，正则语言仅适用于描述词法（如：标识符、实数），而不足以描述表达式、语句等复杂结构（如，括号配对）
- 上下文无关文法在高级程序设计语言的规范说明及其编译技术中有重要应用。
- 我们将讨论“语法分析树”，即利用语法解析语言中的字符串所得到的结构图。分析树是编程语言分析器的产物，是通常获得程序结构的方式。We will discuss the "parse tree", a picture of the structure that a grammar places on the strings of its language. The parse tree is the product of a parser for a programming language and is the way that the structure of programs is normally captured.



编译的各个阶段

# 上下文无关文法 Context-Free Grammar

| 文法类型 | G中任意产生式,<br>$\forall \alpha \rightarrow \beta \in P$                                                   |
|------|--------------------------------------------------------------------------------------------------------|
| 0型   | 无限制                                                                                                    |
| 1型   | $ \beta  \geq  \alpha $                                                                                |
| 2型   | $ \beta  \geq  \alpha $ , 并且 $\alpha \in V$                                                            |
| 3型   | $\alpha \rightarrow \beta$ 均具有形式<br>$A \rightarrow w$<br>$A \rightarrow wB$<br>$A, B \in V, w \in T^+$ |

- 文法  $G=(V, T, P, S)$ 。  
 $V$ ——为 **变量**(variable)的非空有穷集。  
 $T$ ——为终极符(terminal)的非空有穷集。  
 $S$ —— $S \in V$ , 是文法G的**开始符号**(start symbol)  
 $P$ ——为**产生式**(production)的非空有穷集合。
- 上下文无关文法的定义 (见第二章)  
 文法  $G=(V, T, P, S)$  被称为是**上下文无关的**, 如果除了形如  $A \rightarrow \varepsilon$  的产生式之外, 对于  $\forall \alpha \rightarrow \beta \in P, \beta \in (V \cup T)^*$ , 均有  $|\beta| \geq |\alpha|$ , 并且  **$\alpha \in V$**  成立。
- 上下文无关文法的**特点**  
 对于  $\forall A \in V$ , 如果  $A \rightarrow \beta \in P$ , 则无论A出现在句型的任何位置, 我们都可以将A替换成 $\beta$ , 而不考虑A的上下文。
- 目前, 绝大多数高级程序设计语言的语法特征都是上下文无关的, 这使得这些语言的编译系统比较容易实现。

[https://docs.python.org/3/reference/compound\\_stmts.html#the-if-statement](https://docs.python.org/3/reference/compound_stmts.html#the-if-statement)

## 8.1. The `if` statement ¶

The `if` statement is used for conditional execution:

```
if_stmt ::= "if" assignment_expression ":" suite
          ("elif" assignment_expression ":" suite)*
          ["else" ":" suite]
```

It selects exactly one of the suites by evaluating the expressions one by one until one is found to be true (see section [Boolean operations](#) for the definition of true and false); then that suite is executed (and no other part of the `if` statement is executed or evaluated). If all expressions are false, the suite of the `else` clause, if present, is executed.

# 派生树 —— 算术表达式的文法

- 算术表达式的文法

$G_{\text{exp1}}:$

$$\begin{aligned} E &\rightarrow E + T \mid E - T \mid T \\ T &\rightarrow T * F \mid T / F \mid F \\ F &\rightarrow F \uparrow P \mid P \\ P &\rightarrow (E) \mid N(L) \mid \text{id} \\ N &\rightarrow \sin \mid \cos \mid \exp \mid \text{abs} \mid \log \mid \text{int} \\ L &\rightarrow L, E \mid E \end{aligned}$$

考虑算术表达式  
 $x + x / y \uparrow 2$  的派生  
过程

## 语法变量的含义

$E$  —— 表达式 (**E**xpression)  
 $T$  —— 项 (**T**erm)  
 $F$  —— 因子 (**F**actor)  
 $P$  —— 初等量 (**P**rimary)  
 $N$  —— 函数名 (**N**ame of function)  
 $L$  —— 列表 (**L**ist)  
 $\text{id}$  —— 标识符 (**i**dentifier)  
 $\uparrow$  —— 幂运算

$\text{id}$  表示基本运算对象，可以是一个表示变量的标识符，也可以是一个常数。

# 派生树 — 算术表达式的文法

- 算术表达式  $x+x/y \uparrow 2$  的不同派生过程

最左派生

$E \Rightarrow E+T$   
 $\Rightarrow T+T$   
 $\Rightarrow F+T$   
 $\Rightarrow P+T$   
 $\Rightarrow x+T$   
 $\Rightarrow x+T/F$   
 $\Rightarrow x+F/F$   
 $\Rightarrow x+P/F$   
 $\Rightarrow x+x/F$   
 $\Rightarrow x+x/F \uparrow P$   
 $\Rightarrow x+x/P \uparrow P$   
 $\Rightarrow x+x/y \uparrow P$   
 $\Rightarrow x+x/y \uparrow 2$

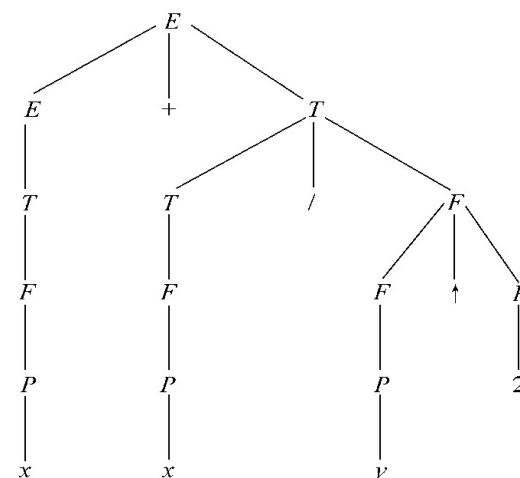
最右派生

$E \Rightarrow E+T$   
 $\Rightarrow E+T/F$   
 $\Rightarrow E+T/F \uparrow P$   
 $\Rightarrow E+T/F \uparrow 2$   
 $\Rightarrow E+T/P \uparrow 2$   
 $\Rightarrow E+T/y \uparrow 2$   
 $\Rightarrow E+F/y \uparrow 2$   
 $\Rightarrow E+P/y \uparrow 2$   
 $\Rightarrow E+x/y \uparrow 2$   
 $\Rightarrow T+x/y \uparrow 2$   
 $\Rightarrow F+x/y \uparrow 2$   
 $\Rightarrow P+x/y \uparrow 2$   
 $\Rightarrow x+x/y \uparrow 2$

随机派生

$E \Rightarrow E+T$   
 $\Rightarrow T+T$   
 $\Rightarrow T+T/F$   
 $\Rightarrow F+T/F$   
 $\Rightarrow F+T/F \uparrow P$   
 $\Rightarrow P+T/F \uparrow P$   
 $\Rightarrow x+T/F \uparrow P$   
 $\Rightarrow x+F/F \uparrow P$   
 $\Rightarrow x+F/P \uparrow 2$   
 $\Rightarrow x+P/P \uparrow 2$   
 $\Rightarrow x+P/y \uparrow 2$   
 $\Rightarrow x+x/y \uparrow 2$

算术表达式  $x+x/y \uparrow 2$  的派生树

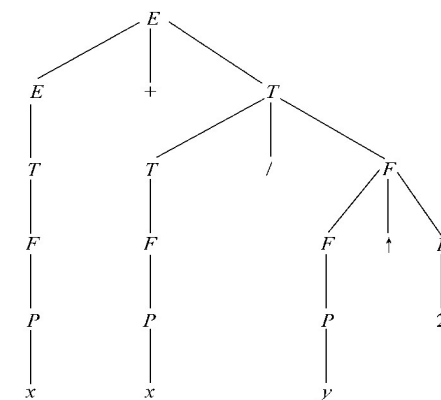


可见，一个句子的派生并不是唯一的。  
为了更清楚、更直观地表达出句子的结构，**可以忽略派生中替换的顺序**，只保留替换的结果。  
**派生树**能够更加直观地表达句子的结构。

# 派生树 Derivation Tree

- 定义6-1：设有CFG  $G=(V, T, P, S)$ ， $G$ 的派生树(derivation tree)是满足如下条件的有序树(ordered tree)

- ① 树的每个顶点有一个标记 $X$ ，且 $X \in V \cup T \cup \{\epsilon\}$
- ② 树根的标记为 $S$ ；
- ③ 如果非叶子顶点 $v$ 标记为 $A$ ， $v$ 的儿子从左到右依次为 $v_1, v_2, \dots, v_n$ ，并且它们分别标记为 $X_1, X_2, \dots, X_n$ ，则 $A \rightarrow X_1 X_2 \dots X_n \in P$ ；
- ④ 如果 $X$ 是一个非叶子顶点的标记，则 $X \in V$ ；
- ⑤ 如果顶点 $v$ 标记为 $\epsilon$ ，则 $v$ 是该树的叶子，并且 $v$ 是其父顶点的唯一儿子。



- 派生树别称
  - 生成树 (derivation tree)
  - 分析树 (parse tree)
  - 语法树 (syntax tree)
- 派生树都是CFG的派生树，因此可以省略CFG限定词。

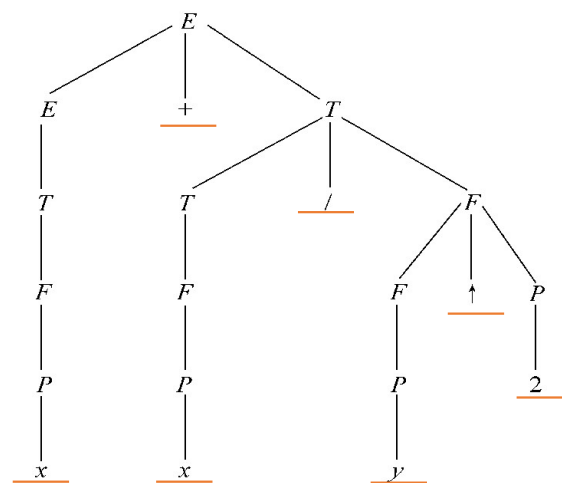
# 派生树 Derivation Tree

- 定义6-2：顶点的顺序

- $v_1, v_2$  是派生树  $T$  的两个不同顶点，如果存在顶点  $v$ ， $v$  至少有两个儿子，使得  $v_1$  是  $v$  的较左儿子的后代， $v_2$  是  $v$  的较右儿子的后代，则称顶点  $v_1$  在顶点  $v_2$  的左边，顶点  $v_2$  在顶点  $v_1$  的右边。

- 定义6-3：派生树的结果 (yield)

- 派生树  $T$  的所有叶子顶点从左到右依次标记为  $X_1, X_2, \dots, X_n$ ，则称符号串  $X_1X_2\dots X_n$  是  $T$  的结果。
- 句型  $\alpha$  的派生树：“结果为  $\alpha$  的派生树”。
- 派生树的结果可以是句子，也可以是句型
- 一个文法可以有多棵派生树，它们可以有不同的结果。

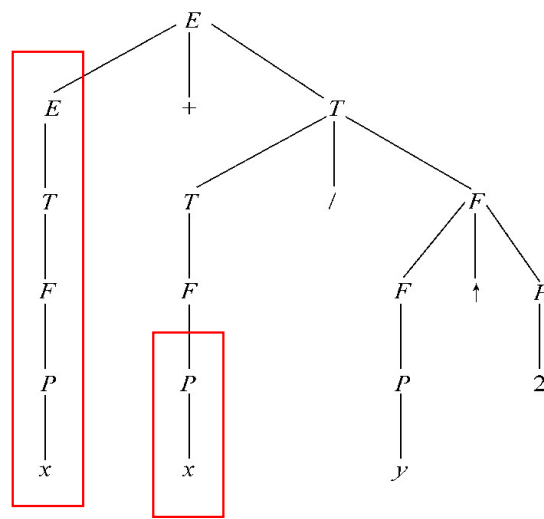


## 定义6-4 派生子树 subtree

- 定义6-4: 派生子树(subtree)
  - 满足派生树定义中除了对根结点的标记的要求以外各条件的树叫派生子树(subtree)。
  - 如果这个子树的根标记为A, 则称之为A子树。
- 定理6-1 设CFG  $G=(V, T, P, S)$ ,  $S \Rightarrow^* \alpha$ 的充分必要条件为G有一棵结果为 $\alpha$ 的派生树。

证明：

(1) 先证明一个更一般的结论：对于任意  $A \in V$ ,  $A \Rightarrow^* \alpha$  的充分必要条件是  $G$  有一棵结果为  $\alpha$  的子树；由于  $S \in V$ , 所以结论对于  $S$  也成立。



# 定理6-1 证明

充分性：如果G有一棵结果为 $\alpha$ 的派生树，则一定有 $S \Rightarrow^* \alpha$ 。

充分性

设G有一棵为 $\alpha$ 的A子树，对A子树的非叶子顶点的个数施行归纳，证明 $A \Rightarrow^* \alpha$ 成立

当 $n=0$ 时，显然成立

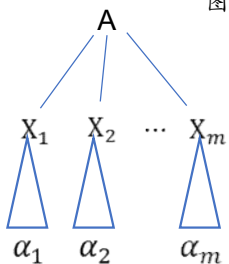
没有非叶子顶点， $A=\alpha$

当 $n=1$ 时

1. A子树是一个二级子树
2. 假设此子树的叶子节点从左到右为 $X_1X_2\ldots X_m$
3. 由定义6-1的第(3)条必有 $A \rightarrow X_1X_2\ldots X_m \in P$
4. 注意到子树的结果为 $\alpha$ ，故 $A \Rightarrow^* \alpha$ ，结论对 $n=1$ 成立

当 $n \leq k$  ( $k \geq 1$ ) 时成立，证明 $n=k+1$ 时成立

图6-3



1. 设A子树有 $k+1$ 个非叶子顶点，根为A的儿子为 $v_1v_2\ldots v_m$ ，并且标记为 $X_1X_2\ldots X_m$
2. 由定义6-1的第(3)条必有 $A \rightarrow X_1X_2\ldots X_m \in P$
3. 设分别以 $X_1X_2\ldots X_m$ 为根的子树的结果为 $\alpha_1\alpha_2\ldots\alpha_m$
4. 显然以 $X_1X_2\ldots X_m$ 的非叶子顶点的个数不大于 $k$
5. 由归纳假设 $X_1 \Rightarrow^* \alpha_1; X_2 \Rightarrow^* \alpha_2; \dots; X_m \Rightarrow^* \alpha_m$ 且 $\alpha = \alpha_1\alpha_2\ldots\alpha_m$
6. 从而 $A \Rightarrow^* X_1X_2\ldots X_m \Rightarrow^* \alpha_1X_2\ldots X_m \Rightarrow^* \alpha_1\alpha_2\ldots X_m \Rightarrow^* \alpha_1\alpha_2\ldots\alpha_m$
7. 即结论对 $n=k+1$ 成立，由归纳法，结论对任意 $n$ 成立

# 定理6-1 证明

必要性：如果有  $S \Rightarrow^* \alpha$ ，则G一定有一棵结果为  $\alpha$  的派生树。

必要性

施行归纳，证明  $A \Rightarrow^* \alpha$  成立  
 设  $A \Rightarrow^* \alpha$ ，施归纳于派生步骤  $n$ ，  
 证明存在结果为  $\alpha$  的A子树

当  $n=0$  时，  
 显然成立

$n=0$ ，表示不用派生， $A=\alpha$

当  $n=1$  时

1. 由  $A \Rightarrow \alpha$  知道  $A \rightarrow \alpha \in P$
2. 令  $\alpha = X_1 X_2 \dots X_m$
3. 则有图6-4所示的A子树
4. 结论对  $n=1$  成立

当  $n \leq k$  ( $k \geq 1$ ) 时成立，  
 证明  $n=k+1$  时成立

1. 令  $A \Rightarrow^{k+1} \alpha$ ，则有  $A \Rightarrow^* X_1 X_2 \dots X_m \Rightarrow^* \alpha_1 X_2 \dots X_m \Rightarrow^* \alpha_1 \alpha_2 \dots X_m \Rightarrow^* \alpha_1 \alpha_2 \dots \alpha_m$   
 其中任意的  $1 \leq i \leq m, X_i \Rightarrow^* \alpha_i$
2. 由归纳假设存在以  $\alpha_i$  为结果的  $X_i$  的子树
3. 由于  $A \Rightarrow X_1 X_2 \dots X_m$ ，所以  $A \rightarrow X_1 X_2 \dots X_m \in P$ ，从而可以得到图6-3所示的A子树的上半部分，然后在将所有的  $X_i$  的子树对应地接在  $X_i$  所标识的节点上，就得到了图6-3所示的树
4. 显然，该树的结果为  $\alpha$ ，所以结论对  $n=k+1$  成立。由归纳法原理，结论对任意的  $n$  成立

图6-3

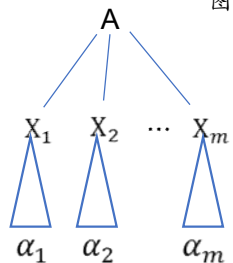
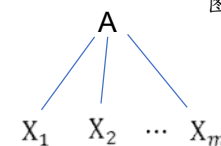
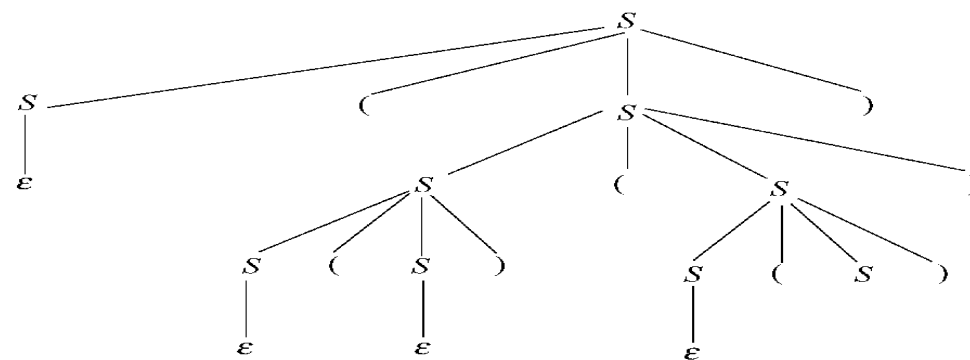


图6-4

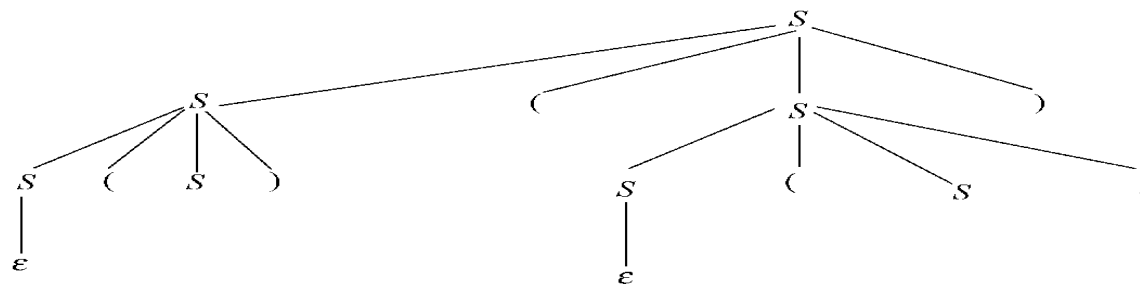


# 例6-1

- 例6-1 设  $G_{\text{bra}}: S \rightarrow S(S) | \epsilon$ , 求  $(O(O))$  和  $(S)((S))$  的派生树。



(a)  $(O(O))$  对应的派生树



(b)  $(S)((S))$  对应的派生树

# 定义6-5

- 定义6-5:

- 最左派生 (leftmost derivation)

$\alpha$ 的派生过程中，每一步都是对当前句型的最左变量进行替换。

- 左句型 (left sentential form)

最左派生得到的句型可叫做左句型。

- 最右归约 (rightmost reduction)

与最左派生相对的归约叫做最右归约。

$G_{\text{exp1}}:$   $E \rightarrow E+T \mid E-T \mid T$   
 $T \rightarrow T * F \mid T / F \mid F$   
 $F \rightarrow F \uparrow P \mid P$   
 $P \rightarrow (E) \mid N(L) \mid \text{id}$   
 $N \rightarrow \sin \mid \cos \mid \exp \mid \text{abs} \mid \log \mid \text{int}$   
 $L \rightarrow L, E \mid E$

$E \Rightarrow E+T$   
 $\Rightarrow T+T$   
 $\Rightarrow F+T$   
 $\Rightarrow P+T$   
 $\Rightarrow x+T$   
 $\Rightarrow x+T/F$   
 $\Rightarrow x+F/F$   
 $\Rightarrow x+P/F$   
 $\Rightarrow x+x/F$   
 $\Rightarrow x+x/F \uparrow P$   
 $\Rightarrow x+x/P \uparrow P$   
 $\Rightarrow x+x/y \uparrow P$   
 $\Rightarrow x+x/y \uparrow 2$  (最左派生)

# 定义6-5

- **最右派生** (rightmost derivation)

$\alpha$  的派生过程中，每一步都是对当前句型的最右变量进行替换。

- **右句型** (right sentential form)

最右派生得到的句型可叫做右句型。

- **最左归约** (leftmost reduction)

与最右派生相对的归约叫做最左归约。

$G_{\text{exp1}}: E \rightarrow E+T \mid E-T \mid T$   
 $T \rightarrow T * F \mid T / F \mid F$   
 $F \rightarrow F \uparrow P \mid P$   
 $P \rightarrow (E) \mid N(L) \mid \text{id}$   
 $N \rightarrow \sin \mid \cos \mid \exp \mid \text{abs} \mid \log \mid \text{int}$   
 $L \rightarrow L, E \mid E$

$E \Rightarrow E+T$   
 $\Rightarrow E+T/F$   
 $\Rightarrow E+T/F \uparrow P$   
 $\Rightarrow E+T/F \uparrow 2$   
 $\Rightarrow E+T/P \uparrow 2$   
 $\Rightarrow E+T/y \uparrow 2$   
 $\Rightarrow E+F/y \uparrow 2$   
 $\Rightarrow E+P/y \uparrow 2$   
 $\Rightarrow E+x/y \uparrow 2$   
 $\Rightarrow T+x/y \uparrow 2$   
 $\Rightarrow F+x/y \uparrow 2$   
 $\Rightarrow P+x/y \uparrow 2$   
 $\Rightarrow x+x/y \uparrow 2$  (最右派生)

# 定义6-5

---

- **规范派生**(normal derivation)
  - 最右派生。
  - 一般地，由于计算机在处理一个输入串时，通常都是从左到右进行的，这使得对句型的分析按照从左到右的顺序是比较自然的。所以最右派生又称为规范派生。
- **规范句型**(normal sentential form)
  - 规范派生产生的句型。
- **规范归约**(normal reduction)
  - 最左归约。

# 最左派生和最右派生

---

- 定理6-2 如果 $\alpha$ 是CFG  $G$ 的一个句子，则 $G$ 中存在 $\alpha$ 的最左派生和最右派生。  
证明（略）
- 定理6-3 如果 $\alpha$ 是CFG  $G$ 的一个句子， $\alpha$ 的派生树与最左派生和最右派生是一一对应的，但是，这棵派生树可以对应多个不同的派生。

# 二义性 Ambiguity

- 简单算术表达式的二义性文法

$G_{\text{exp2}}: E \rightarrow E + E \mid E - E \mid E / E \mid E * E$

$E \rightarrow E \uparrow E \mid (E) \mid N(L) \mid \text{id}$

$N \rightarrow \sin \mid \cos \mid \exp \mid \text{abs} \mid \log \mid \text{int}$

$L \rightarrow L, E \mid E$

考察句型  
 $x + x / y \uparrow 2$   
和派生树之间  
的关系?

$E \Rightarrow E + E$   
 $\Rightarrow x + E$   
 $\Rightarrow x + E / E$   
 $\Rightarrow x + x / E$   
 $\Rightarrow x + x / E \uparrow E$   
 $\Rightarrow x + x / y \uparrow E$   
 $\Rightarrow x + x / y \uparrow 2$

句子表达的意思是x加上x/y<sup>2</sup>

$E \Rightarrow E / E$   
 $\Rightarrow E + E / E$   
 $\Rightarrow x + E / E$   
 $\Rightarrow x + x / E$   
 $\Rightarrow x + x / E \uparrow E$   
 $\Rightarrow x + x / y \uparrow E$   
 $\Rightarrow x + x / y \uparrow 2$

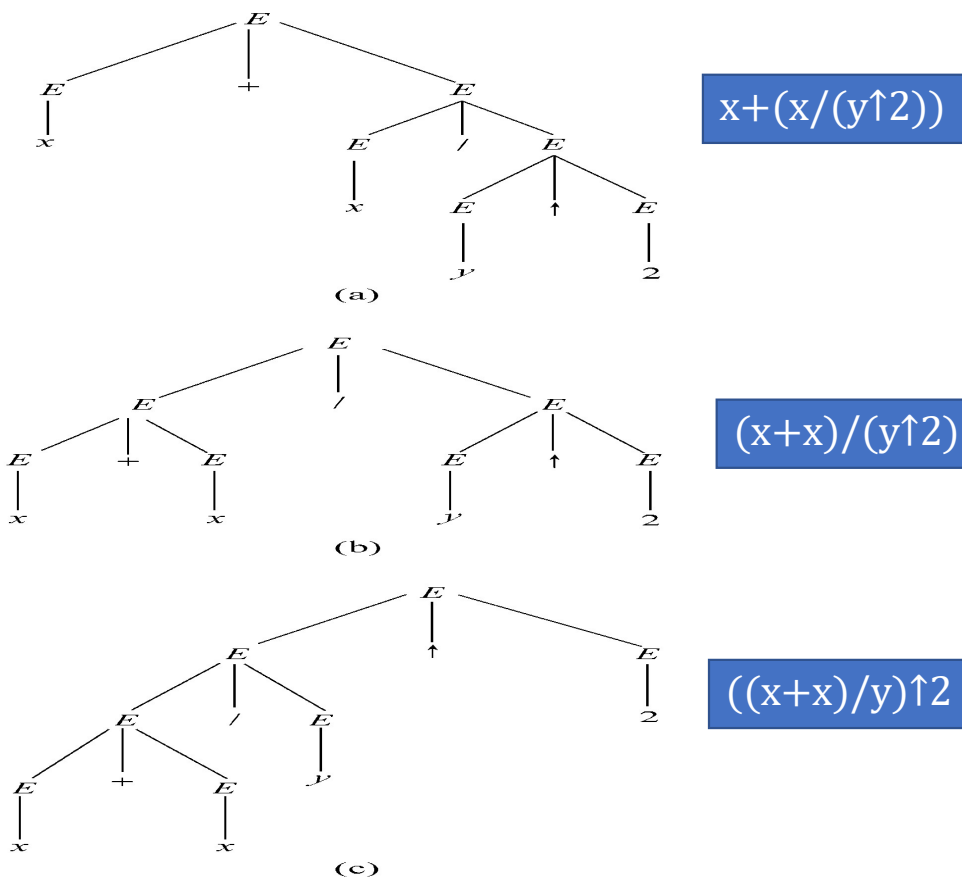
句子表达的意思是x+x除以y<sup>2</sup>

$E \Rightarrow E \uparrow E$   
 $\Rightarrow E / E \uparrow E$   
 $\Rightarrow E + E / E \uparrow E$   
 $\Rightarrow x + E / E \uparrow E$   
 $\Rightarrow x + x / E \uparrow E$   
 $\Rightarrow x + x / y \uparrow E$   
 $\Rightarrow x + x / y \uparrow 2$

句子表达的意思是((x+x)/y)<sup>2</sup>

# 二义性 Ambiguity

3个不同的最左派生过程对应3个不同的派生树



不同的最左派生（派生树）表达了句子（句型）的不同含义，句子的多义性会给语言的分析带来麻烦，一般在利用文法定义语言或者对语言进行分析时，不希望出现这种情况。

# 二义性 Ambiguity

## 定义6-6 二义性(ambiguity)

- 设有CFG  $G=(V, T, P, S)$ , 如果存在  $w \in L(G)$ ,  $w$  至少有两棵不同的派生树, 则称  $G$  是二义性的。否则,  $G$  为非二义性的。

$G_{\text{exp1}}:$

$E \rightarrow E+T | E-T | T$

$T \rightarrow T * F | T / F | F$

$F \rightarrow F \uparrow P | P$

$P \rightarrow (E) | N(L) | \text{id}$

$N \rightarrow \text{sin} | \text{cos} | \text{exp} | \text{abs} | \text{log} | \text{int}$

$L \rightarrow L, E | E$

$G_{\text{exp2}}:$

$E \rightarrow E+E | E-E | E/E | E * E$

$E \rightarrow E \uparrow E | (E) | N(L) | \text{id}$

$N \rightarrow \text{sin} | \text{cos} | \text{exp} | \text{abs} | \text{log} | \text{int}$

$L \rightarrow L, E | E$

能否判断给  
定文法是否  
具有二义性?

不可解问题!

$G_{\text{exp1}}$  与  $G_{\text{exp2}}$  是等价文法, 即  $L(G_{\text{exp2}}) = L(G_{\text{exp1}})$ , 其中文法  $G_{\text{exp2}}$  是二义的, 而文法  $G_{\text{exp1}}$  派生的句子  $x+x/y \uparrow 2$  是唯一的, 即  $G_{\text{exp1}}$  是无二义性的。可见, 一个语言, 在产生它的众多文法中, 有的可能是具有二义性的, 有的是无二义性的。

## 例 6-2 消除二义性

例6-2 为消除 $G_{ifa}$ 的二义性对文法进行改造。

$G_{ifa}$ :  $S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S$

**Eg:**  $\text{if } a \text{ then if } b \text{ then } s \text{ else } s2$

可解释为 $\text{if } a \text{ then (if } b \text{ then } s) \text{ else } s2$

或解释为 $\text{if } a \text{ then (if } b \text{ then } s \text{ else } s2)$

$G_{ifm}$ :  $S \rightarrow U \mid M$

$U \rightarrow \text{if } E \text{ then } S$

$U \rightarrow \text{if } E \text{ then } M \text{ else } U$

$M \rightarrow \text{if } E \text{ then } M \text{ else } M \mid S$

$G_{ifh}$ :  $S \rightarrow TS \mid CS$

$C \rightarrow \text{if } E \text{ then}$

$T \rightarrow CS \text{ else}$

$G_{ifa}$ 是二义性文法，这里的问题是else与哪一个if进行匹配的问题。

$G_{ifm}$ 是为了消除二义性而进行的文法改造。

$M$ 表示一个匹配的if语句（即含else）， $U$ 表示一个非匹配的if语句（即不含有else）

该文法除了迫使then和else之间必须有一个匹配的语句之外，还需要执行**最长匹配原则**才能完全消除二义性  
另外，也可以要求它的 $M$ 产生式的**第二个候选式 $S$ 表示赋值语句、循环语句等非if语句**，以此来避免一个 $U$ 被规约为 $M$ 。

$G_{ifh}$ 是为了消除二义性而进行的另一种文法改造方法。  
在分析中需要执行**最长匹配原则**才能完全消除二义性，即没有完全消除二义性。

- 综上，需要彻底消除二义性，还需要使用其他的一些限制，如“**最长匹配原则**”和“ **$M$ 的第二个候选式 $S$ 是非if语句**”等文法本身并未表达出来的限制。
- 解决二义性的方法还有：**给匹配确定优先级，增加标志等**。

## 例 6-3 固有二义性的语言

是否所有语言都有对应的非二义性文法呢？

•例 6-3 设

$$L_{\text{ambiguity}} = \{0^n 1^n 2^m 3^m \mid n, m \geq 1\} \cup \{0^n 1^m 2^m 3^n \mid n, m \geq 1\}$$

可以用如下文法产生语言  $L_{\text{ambiguity}}$ ：

G:  $S \rightarrow AB \mid 0C3$

$A \rightarrow 01 \mid 0A1$

$B \rightarrow 23 \mid 2B3$

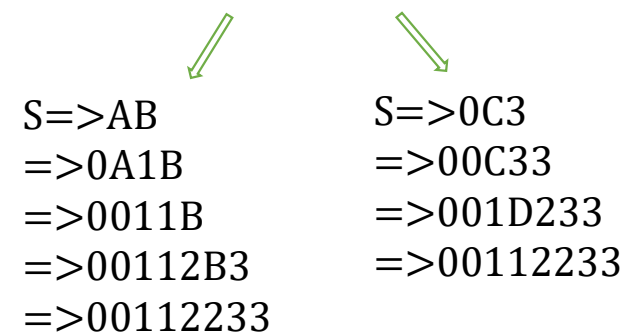
$C \rightarrow 0C3 \mid 12 \mid 1D2$

$D \rightarrow 12 \mid 1D2$

可以发现， $L_{\text{ambiguity}}$  中形如  $0^n 1^n 2^n 3^n$  的句子都有不同的派生树存在。

语言  $L_{\text{ambiguity}}$  不存在非二义性的文法。

不难找到句子 00112233  
两个不同的最左派生



## 定义 6-7 固有二义性的语言

---

- 定义6-7：固有二义性的(inherent ambiguity)
  - 如果语言L不存在非二义性文法，则称L是固有二义性的，又称L是先天二义性的。
  - 文法可以是二义性的，语言可以是固有二义性的。
  - 一般地，不说文法是固有二义性的，也不说语言是二义性的。

# 基于CFG的分析 Analysis base on CFG

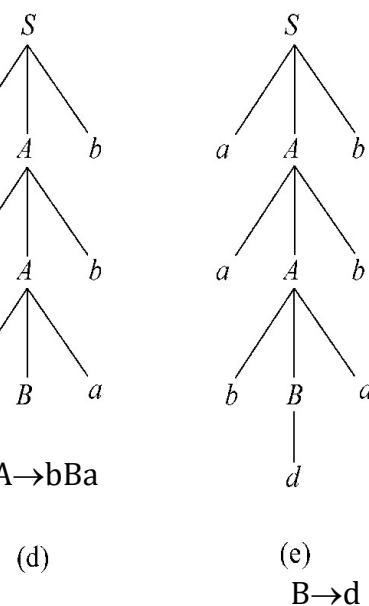
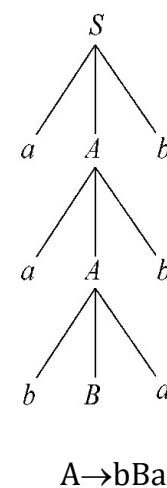
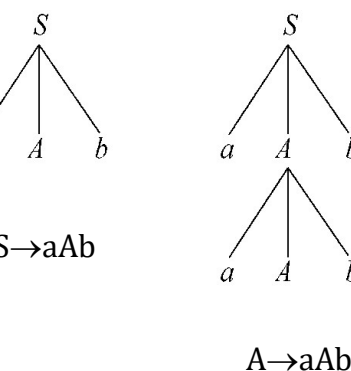
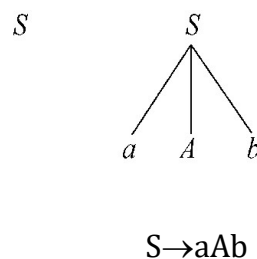
- 自顶向下的分析方法

- 通过考察是否可以从给定文法的开始符号派生出一个符号串，可以判定一个符号串是否为该文法的句子。

- 例

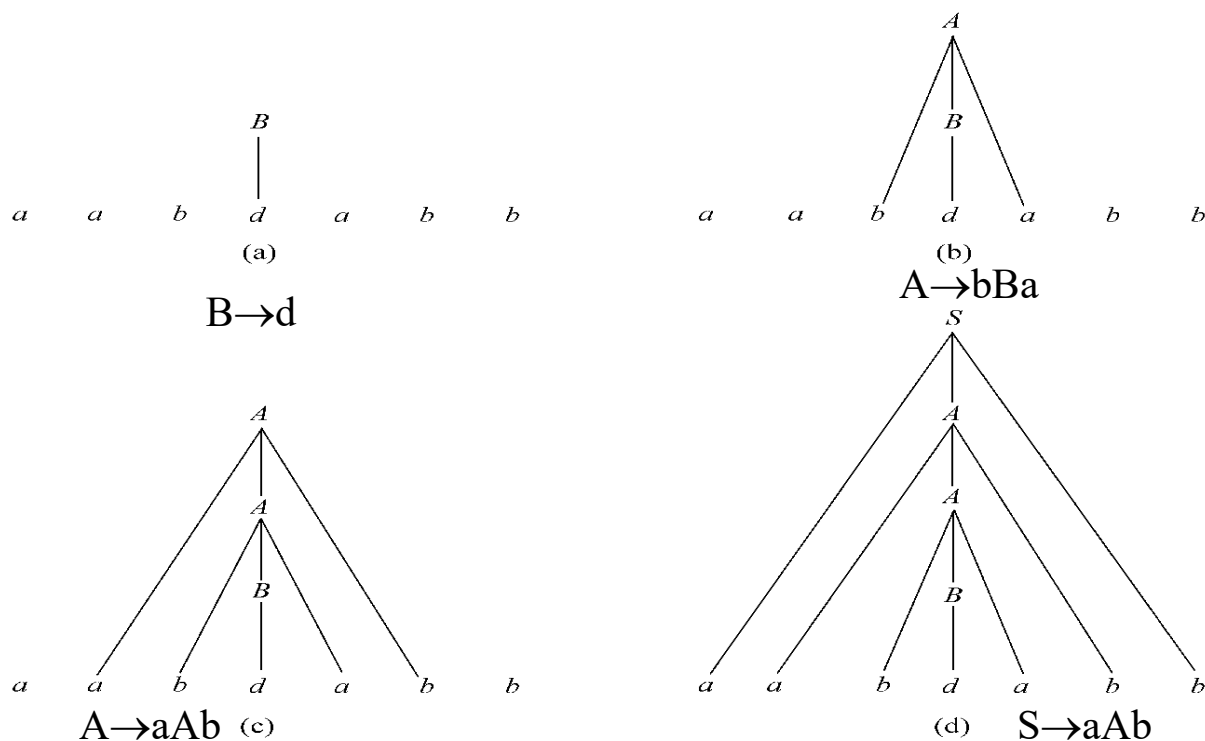
- $S \rightarrow aAb | bBa$
- $A \rightarrow aAb | bBa$
- $B \rightarrow d$

判断aabdbabb是否是该文法的句子



# 基于CFG的分析 Analysis base on CFG

- 自底向上的分析方法
  - 通过考察是否可以将一个符号串归约为给定文法的开始符号，完成判定一个符号串是否为该文法的句子的任务。



## 6.2 CFG的化简

考虑表示语言的文法

$$L(G) = \{0x \mid x \in \{0,1\}^*\} \cup \{0x_y \mid x \in \{0,1\}^*, y \in \{0,1\}^+\}$$

$G_1$ :  $S \rightarrow 0 \mid 0A \mid E$   
 $A \rightarrow \varepsilon \mid 0A \mid 1A \mid B$   
 $B \rightarrow \_C$   
 $C \rightarrow 0 \mid 1 \mid 0C \mid 1C$   
 $D \rightarrow 1 \mid 1D \mid 2D$   
 $E \rightarrow 0E2 \mid E02$

文法构造有什么问题吗?

$G_2$ :  $S \rightarrow 0 \mid 0A$   
 $A \rightarrow \varepsilon \mid 0A \mid 1A \mid B$   
 $B \rightarrow \_C$   
 $C \rightarrow 0 \mid 1 \mid 0C \mid 1C$

去掉D、E

$G_3$ :  $S \rightarrow 0 \mid 0A$   
 $A \rightarrow 0 \mid 1 \mid 0A \mid 1A \mid B$   
 $B \rightarrow \_C$   
 $C \rightarrow 0 \mid 1 \mid 0C \mid 1C$

去掉  $A \rightarrow \varepsilon$

去掉  $A \rightarrow B$

$G_4$ :  $S \rightarrow 0 \mid 0A$   
 $A \rightarrow 0 \mid 1 \mid 0A \mid 1A \mid \_C$   
 $C \rightarrow 0 \mid 1 \mid 0C \mid 1C$

结论：文法中出现的符号以及所使用的产生式不一定是恰当的，对文法进行化简是支持后续分析的重要环节！

# 定义6-8 无用符号 Useless Symbol

- 定义6-8：无用符号(useless symbol)

- 对CFG  $G=(V, T, P, S)$ ，任意  $X \in V \cup T$ ，如果存在  $w \in L(G)$ ， $X$  出现在  $w$  的派生过程中，即存在  $\alpha, \beta \in (V \cup T)^*$ ，使得  $S \Rightarrow^* \alpha X \beta \Rightarrow^* w$ ，则称  $X$  是有用的，否则，称  $X$  是**无用符号**。

理解无用符号：

(1)  $G$  中的符号  $X$  既可能是有用的，也可能是无用的。当  $X$  是无用的时候，它既**可能是终结符号，也可能是语法变量**。所以在消除文法中的无用符号过程中，**应逐个考察  $V \cup T$**  中的所有符号。

(2) 对于任意  $X \in V \cup T$ ，如果  $X$  是有用的，它必须同时满足如下两个条件：  
① 存在  $w \in T^*$ ，使得  $X \Rightarrow^* w$ ；  
②  $\alpha, \beta \in (V \cup T)^*$ ，使得  $S \Rightarrow^* \alpha X \beta$ 。

(3) 注意到文法是语言的有穷描述，所以，集合  $V, T, P$  都是有穷的。从而我们有可能构造出有效的算法，来完成消除文法的无用符号的工作。

# 算法 6-1 去除无用符号 (1)

算法 6-1 删除派生不出终极符号行的变量。

- 输入: CFG  $G=(V, T, P, S)$ 。
- 输出: CFG  $G'=(V', T, P', S)$ ,  $V'$  中不含派生不出终极符号行的变量, 并且  $L(G')=L(G)$ 。
- 算法步骤:

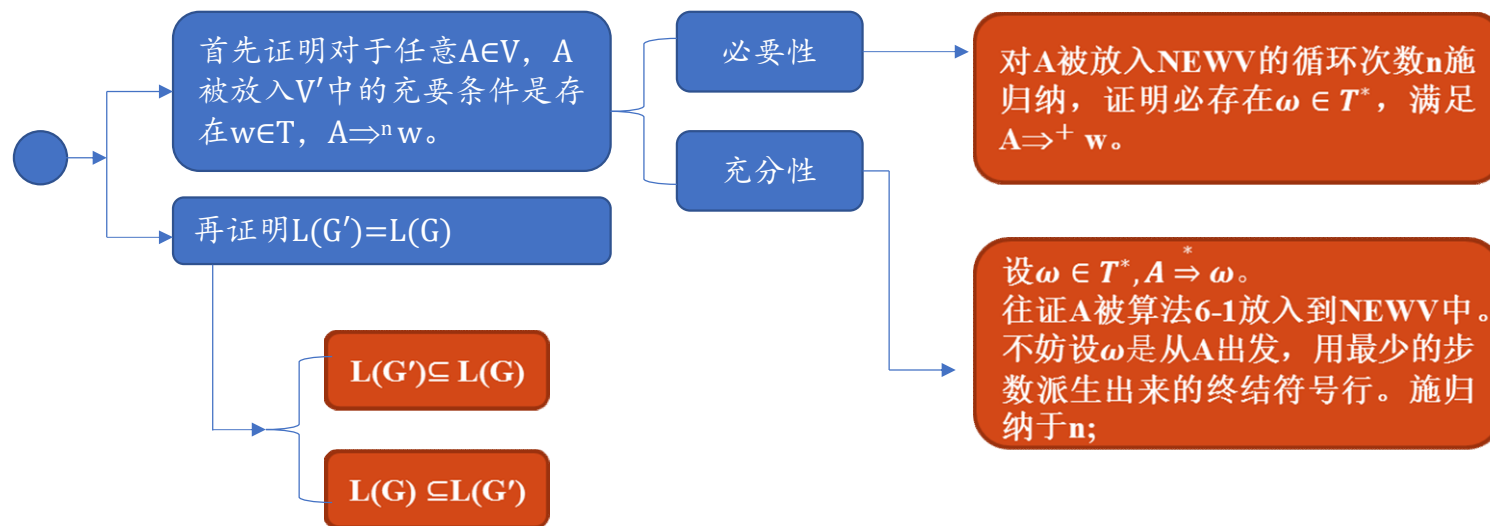
```
(1) OLDV= $\Phi$ ; //保存能推出终极符号行的变量
(2) NEWV= $\{A|A\rightarrow w\in P \text{ 且 } w\in T^*\}$ ; //首先能直接推出终极符号行的变量是有用符号
(3) while OLDV $\neq$ NEWV do //NEWV增加了新变量就要再循环计算
    begin
(4)     OLDV=NEWV; //将识别到的有用符号放入OLDV中
(5)     NEWV=OLDV $\cup \{A|A\rightarrow \alpha\in P \text{ and } \alpha\in (T\cup OLDV)^*\}$  //能够推出终结符和已知的有用符号的句型的变量, 也是有用符号
    end

(6)  $V'$ =NEWV; //NEWV就是 $G'$ 的变量集合
(7)  $P'=\{A\rightarrow \alpha| A\rightarrow \alpha\in P \text{ 且 } A\in V' \text{ 且 } \alpha\in (T\cup V')^*\}$  //所有有用符号相关的产生式构成的集合是 $G'$ 的产生式集合
```

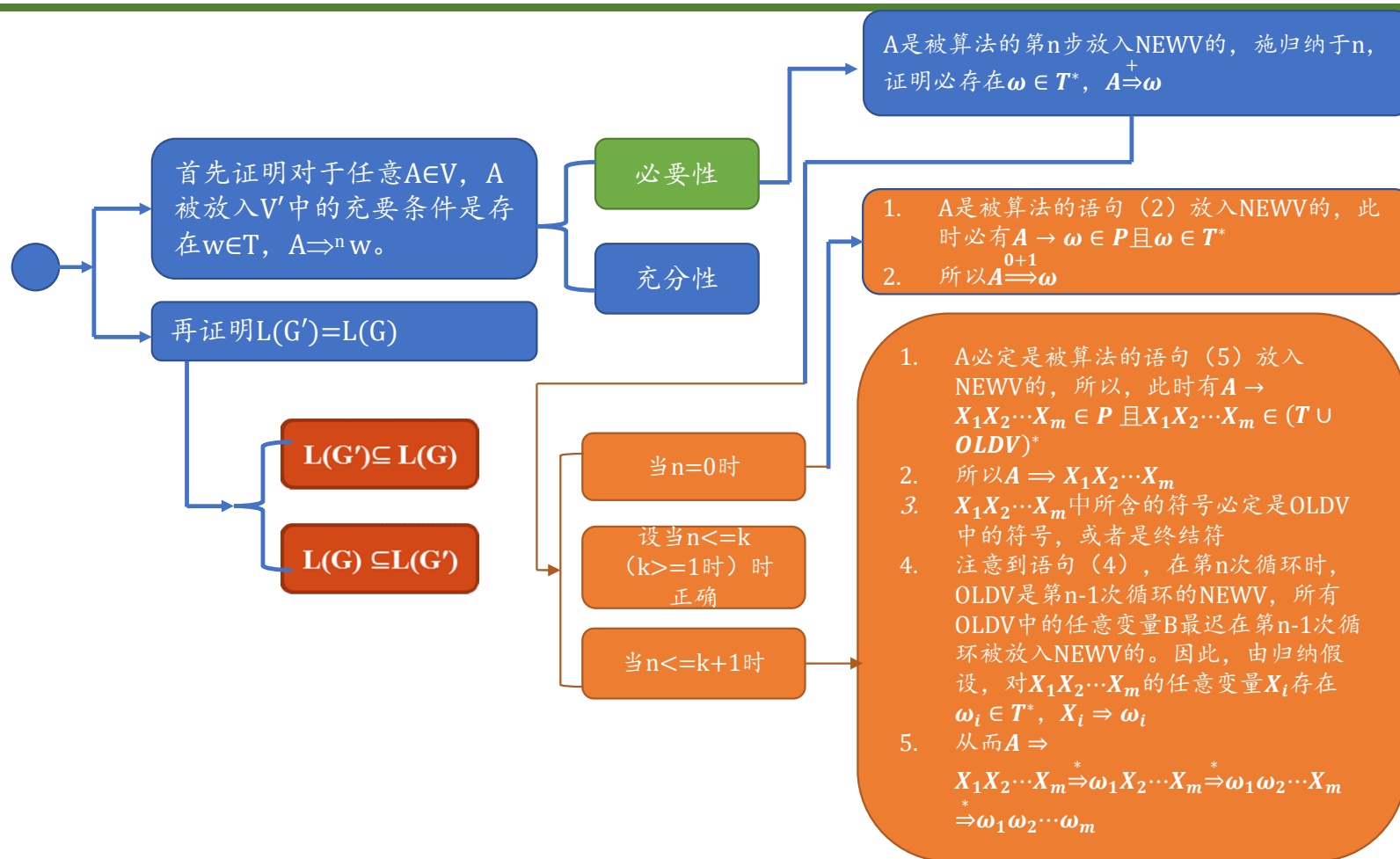
$G_1$ :  $S\rightarrow 0|0A|E$   
 $A\rightarrow \varepsilon|0A|1A|B$   
 $B\rightarrow \_C$   
 $C\rightarrow 0|1|0C|1C$   
 $D\rightarrow 1|1D|2D$   
 $E\rightarrow 0E2|E02$

# 定理 6-4 证明算法6-1的正确性

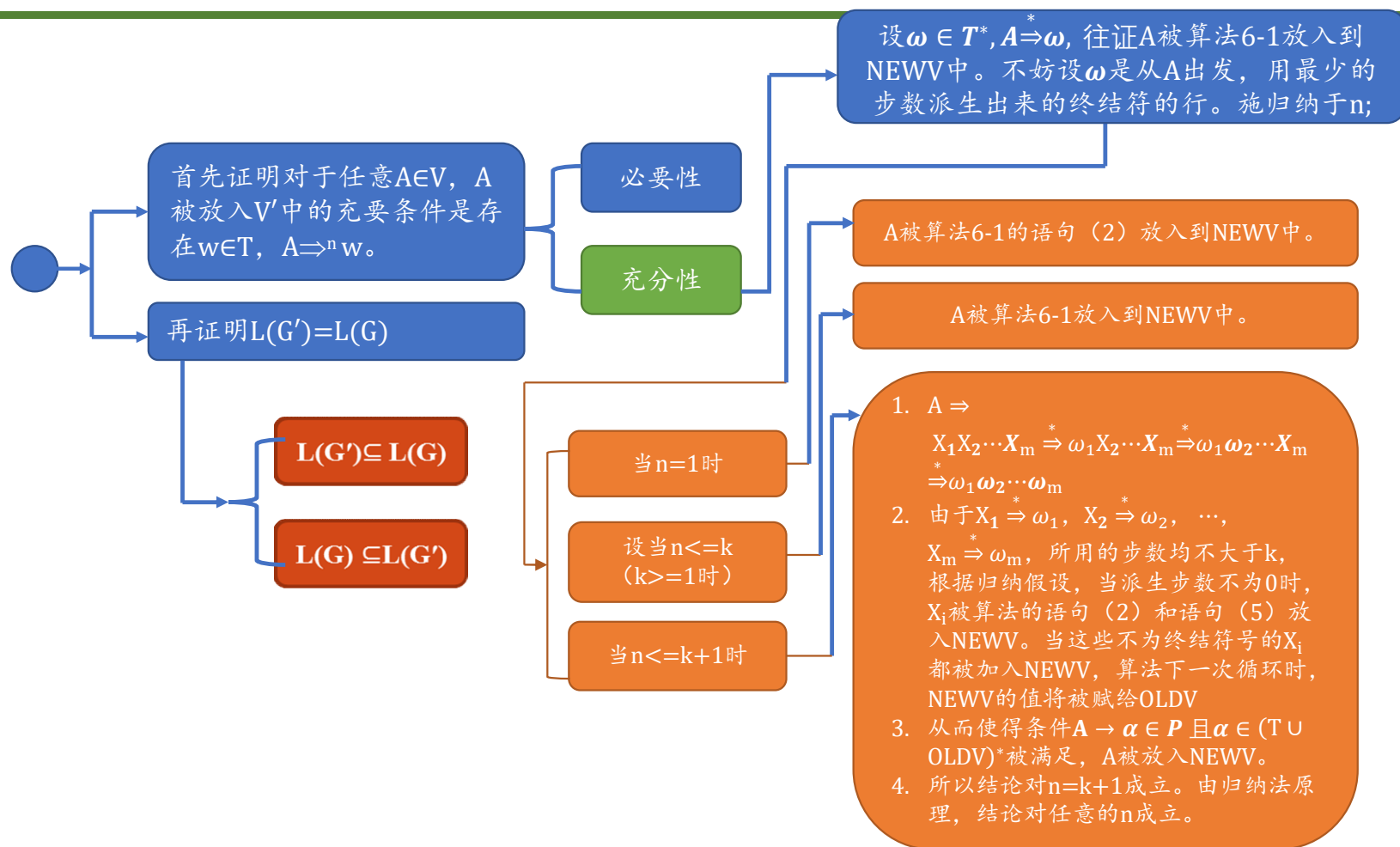
定理 6-4 算法6-1是正确的。



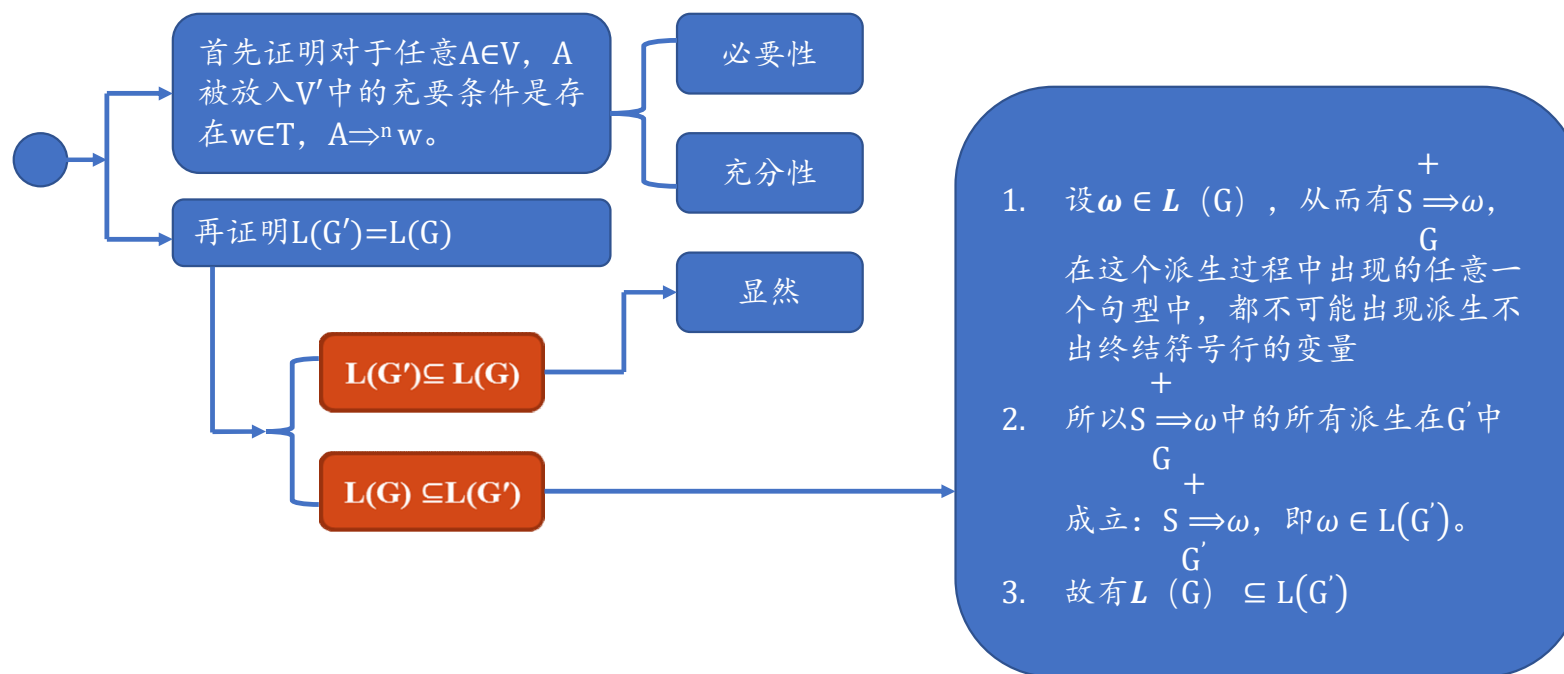
# 定理 6-4 证明算法6-1的正确性



# 定理 6-4 证明算法6-1的正确性



# 定理 6-4 证明算法6-1的正确性



## 算法 6-2 去除无用符号 (2)

算法 6-2 删除不出现在任何句型中的语法符号。  
(包括变量和终结符)

• 输入:

CFG  $G=(V, T, P, S)$ 。

• 输出:

CFG  $G'=(V', T', P', S)$ ,  $V' \cup T'$  中的符号必在  $G$  的某个句型中出现, 并且有  $L(G')=L(G)$ 。

$G_1$ :  $S \rightarrow 0|0A|E$   
 $A \rightarrow \varepsilon|0A|1A|B$   
 $B \rightarrow \_C$   
 $C \rightarrow 0|1|0C|1C$   
 $D \rightarrow 1|1D|2D$   
 $E \rightarrow 0E2|E02$

算法步骤:

(1)  $OLDV = \Phi$ ;

(2)  $OLDT = \Phi$ ;

(3)  $NEWV = \{S\} \cup \{A | S \rightarrow \alpha A \beta \in P\}$ ;

(4)  $NEWT = \{a | S \rightarrow \alpha a \beta \in P\}$ ;

(5) while  $OLDV \neq NEWV$  或者  $OLDT \neq NEWT$  do  
begin

(6)  $OLDV = NEWV$ ;

(7)  $OLDT = NEWT$ ;

(8)  $NEWV = OLDV \cup \{B | A \in OLDV \text{ and } A \rightarrow \alpha B \beta \in P \text{ 且 } B \in V\}$ ;

(9)  $NEWT = OLDV \cup \{a | A \in OLDV \text{ and } A \rightarrow \alpha a \beta \in P \text{ 且 } a \in T\}$ ;

end

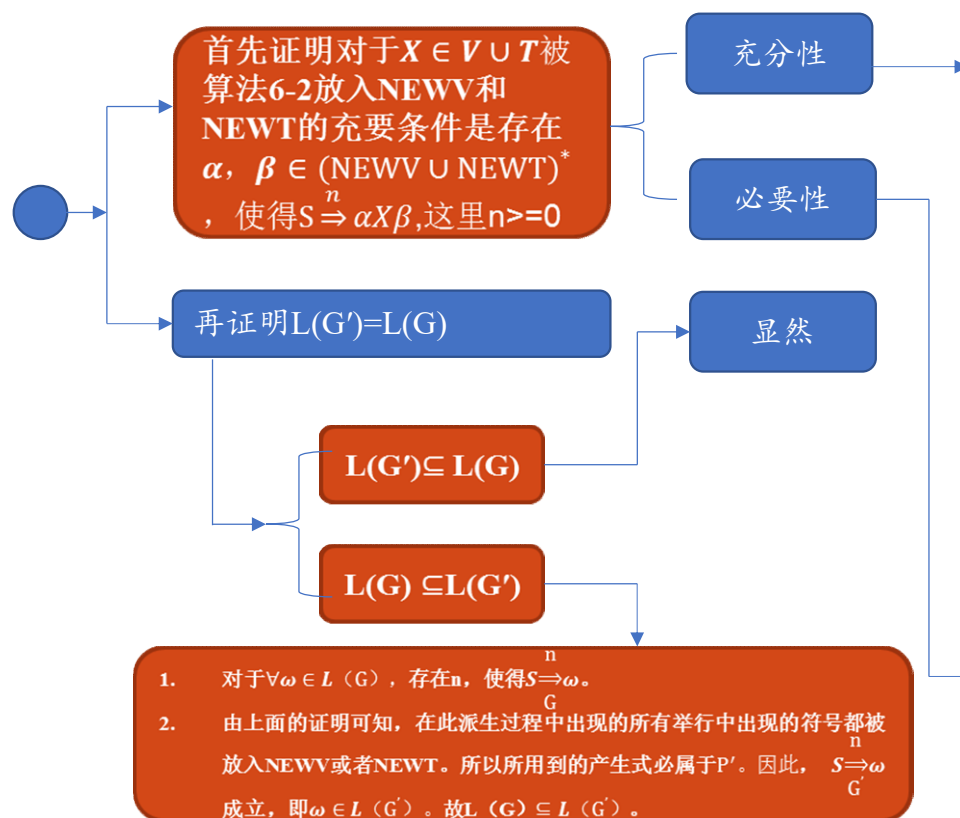
(10)  $V' = NEWV$ ;

(11)  $T' = NEWT$ ;

(12)  $P' = \{A \rightarrow \alpha | A \rightarrow \alpha \in P \text{ 且 } A \in V' \text{ 且 } \alpha \in (T' \cup V')^*\}$ 。

# 定理 6-5 证明算法6-2的正确性

定理 6-5 算法6-2是正确的。



1. 设 $S \Rightarrow^n \alpha X \beta$ , 施归纳于 $n$ ;
2. 当 $n=0$ 时,  $S=X$ ,  $X$ 在算法6-2的语句(3)被放在NEWV;
3. 当 $n=1$ 时, 必定存在 $S \rightarrow \alpha X \beta \in P$ , 所以, 当 $X \in T$ 时, 它被算法6-2的语句(4)放在NEWT, 当 $X \in V$ 时, 它被算法6-2的语句(5)放在NEWV。
4. 设当 $n \leq k$ 时结论成立, 则当 $n=k+1$ 时, 有 $S \Rightarrow^n \alpha_1 A \beta_2 \Rightarrow \alpha_1 \alpha_2 X \beta_1 \beta_2$ , 这里 $\alpha_1 \alpha_2 = \alpha$ ,  $\beta_1 \beta_2 = \beta$
5. 由归纳假设,  $A$ 必定被放入NEWV。则算法6-2在随后的循环中, 含有 $A$ 的NEWV被赋值于OLDV, 从而在本次循环中, 当 $X \in V$ 时, 条件 $A \in OLDV$ 且 $A \rightarrow \alpha X \beta \in P$ 成立, 它被算法6-2的语句(8)放在NEWV; 当 $X \in T$ 时, 条件 $A \in OLDV$ 且 $A \rightarrow \alpha X \beta \in P$ 成立, 它被算法6-2的语句(9)放在NEWT;
6. 所以结论对 $n=k+1$ 成立。由归纳法原理, 结论对任意的 $n$ 成立。

1. 设 $X$ 是在算法6-2的第 $n$ 次循环中被放入NEWT或NEWV的, 对 $n$ 施归纳, 采用与定理6-1的证明相应部分的类似讨论, 易证结论成立

## 定理 6-6 不含无用符号的CFG

定理6-6：对于任意CFL  $L$ ， $L \neq \Phi$ ，则存在不含无用符号的CFG  $G$ ，使得  $L(G)=L$ 。

- 证明要点：依次用算法6-1和算法6-2对文法进行处理，可以得到等价的不含无用符号的文法。
- 注意：不可先用算法6-2后用算法6-1。

例 6-2-1 删除如下文法的无用符号：

$S \rightarrow AB|a|BB$

$A \rightarrow a$

$C \rightarrow b|Aba$

先用算法6-1，文法被化简成：

$S \rightarrow a, A \rightarrow a, C \rightarrow b|Aba$

再用算法6-2，可得到文法：

$S \rightarrow a$



先用算法6-2，文法被化简成：

$S \rightarrow AB|a|BB, A \rightarrow a$

再用算法6-1，可得到文法：

$S \rightarrow a, A \rightarrow a$



显然，该文法中的变量A是新的无用变量，未被删除。

如果先用算法6-1，就不会出现这种情况。

# $\epsilon$ -产生式 $\epsilon$ -production

- 定义6-9:  $\epsilon$ -产生式 ( $\epsilon$ -production)
  - 形如 $A \rightarrow \epsilon$ 的产生式叫做 $\epsilon$ -产生式, 又称为空产生式 (null production)。
- 可空(nullable)变量
  - 对于文法 $G=(V, T, P, S)$ 中的任意变量 $A$ , 如果 $A \Rightarrow^+ \epsilon$ , 则称 $A$ 为可空变量。

- 虽然 $\epsilon$ -产生式对构造文法带来很大的方便, 但是, 它使得对文法所定义的语言的分析变得比较复杂, 在投入分析前, 可以考虑可以消除所有的 $\epsilon$ -产生式;
- 而当 $\epsilon$ 属于 $L(G)$ 时, 可以先构造出 $L(G)-\{\epsilon\}$ 的不含 $\epsilon$ -产生式的文法, 然后加入 $S \rightarrow \epsilon$ 就可以了。

为什么要删除  
 $\epsilon$ -production?

# $\varepsilon$ -产生式 $\varepsilon$ -production

- 分析:
- 假设文法中有以下类型产生式组:
$$A \rightarrow X_1 X_2 \dots X_m$$
$$X_i \rightarrow \varepsilon$$
$$X_j \rightarrow \varepsilon$$
$$X_k \rightarrow X_i X_j$$
- 目标: 在文法表达能力一致的情况下, 将 $\varepsilon$ -产生式从文法中删除。

(1) 首先需要判断哪些变量可能推导出 $\varepsilon$ ——识别可空变量集 $U$

(2) 对形如 $A \rightarrow X_1 X_2 \dots X_m$ 的产生式进行考察, 对 $\forall H \subseteq U$ , 从产生式 $A \rightarrow X_1 X_2 \dots X_m$ 中删除 $H$ 中的变量。

(3) 对于不同的 $H$ , 得到不同的 $A$ 产生式, 用这组 $A$ 产生式替代产生式 $A \rightarrow X_1 X_2 \dots X_m$ 。

(4) 必须避免在这个过程中产生新的 $\varepsilon$ -产生式: 当 $\{X_1, X_2, \dots, X_m\} \subseteq U$ 时, 不可将 $X_1, X_2, \dots, X_m$ 同时从产生式 $A \rightarrow X_1 X_2 \dots X_m$ 中删除。

## 算法 6-3 求CFG G的可空变量集

算法6-3 求CFG G的可空变量集U。

- 输入：CFG  $G=(V, T, P, S)$ 。
- 输出：G的可空变量集U。
- 主要步骤：

```
(1) OLDU= $\Phi$ ;  
(2) NEWU={A|  $A \rightarrow \epsilon \in P$ };  
(3) while NEWU $\neq$ OLDU do  
    begin  
(4)     OLDU = NEWU;  
(5)     NEWU= OLDU  $\cup \{A|A \rightarrow \alpha \in P \text{ and } \alpha \in \text{OLDU}^*\}$   
    end  
(6) U=NEWU
```

# 定理 6-7 去除 $\varepsilon$ -产生式

定理 6-7 对于任意CFG  $G$ ，存在不含 $\varepsilon$ -产生式的CFG  $G'$ 使得 $L(G')=L(G)-\{\varepsilon\}$ 。

证明：

(1) 构造

- 设CFG  $G=(V, T, P, S)$ ,
- 用算法6-3求出 $G$ 的可空变量集 $U$ ,
- 构造 $P'$ 。

对于  $\forall A \rightarrow X_1 X_2 \dots X_m \in P$

将 $A \rightarrow \alpha_1 \alpha_2 \dots \alpha_m$ 放入 $P'$ ，其中

if  $X_i \in U$  then  $\alpha_i = X_i$  或者  $\alpha_i = \varepsilon$ ;

if  $X_i \notin U$  then  $\alpha_i = X_i$

- 要求：在同一产生式中， $\alpha_1, \alpha_2, \dots, \alpha_m$ 不能同时为 $\varepsilon$ 。
- 后续证明 $L(G')=L(G)-\{\varepsilon\}$ （略）

理解：

- (1) 对于  $P$  中，所有非 $\varepsilon$ -产生式进行处理；
- (2) 将各种符合上述代换的产生式均加入 $P'$ ；
- (3) 如果某产生式右侧包含多个变量，则这些变量不能同时为空。

# 例子 去除 $\varepsilon$ -产生式

- 例：消除下面文法中的 $\varepsilon$ -产生式。

$S \rightarrow ABCD$

$A \rightarrow BD|aa|\varepsilon$

$B \rightarrow aB|a$

$C \rightarrow DC|c|\varepsilon$

$D \rightarrow d$

- (1)求可空变量集 $U=\{A, C\}$

- (2)构造新文法

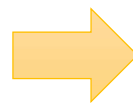
$S \rightarrow ABCD|\varepsilon BCD|AB\varepsilon D|\varepsilon B\varepsilon D$

$A \rightarrow BD|aa$

$B \rightarrow aB|a$

$C \rightarrow DC|c|D\varepsilon$

$D \rightarrow d$



$S \rightarrow ABCD|BCD|ABD|BD$

$A \rightarrow BD|aa$

$B \rightarrow aB|a$

$C \rightarrow DC|c|D$

$D \rightarrow d$

## 定义6-10 单一产生式 Unit Production

- 定义6-10：单一产生式(unit production) :形如 $A \rightarrow B$ 的产生式称为单一产生式。其中 $A, B \in V$ 。

举例：

文法 $G_{\text{exp1}}$ ：

$$\begin{aligned} E &\rightarrow E+T \mid E-T \mid \mathbf{T} \\ T &\rightarrow T * F \mid T / F \mid \mathbf{F} \\ F &\rightarrow F \uparrow P \mid \mathbf{P} \\ P &\rightarrow (E) \mid N(L) \mid \text{id} \\ N &\rightarrow \sin \mid \cos \mid \exp \mid \text{abs} \mid \log \mid \text{int} \\ L &\rightarrow L, E \mid \mathbf{E} \end{aligned}$$

存在派生：

$$E \Rightarrow T \Rightarrow F \Rightarrow P \Rightarrow \text{id}$$

# 定义6-10 单一产生式 Unit Production

$G_{\text{exp3}}: E \rightarrow E+T | E-T | T^*F | T/F | F\uparrow P | (E) | N(L) | \text{id}$

用T产生式组的右部  
替换单一产生式

$T \rightarrow T^*F | T/F | F\uparrow P | (E) | N(L) | \text{id}$

$F \rightarrow F\uparrow P | (E) | N(L) | \text{id}$

用F产生式组的右部替换单一  
产生式

$P \rightarrow (E) | N(L) | \text{id}$

$N \rightarrow \sin | \cos | \exp | \text{abs} | \log | \text{int}$

用P产生式组的右部替换单一产  
生式

$L \rightarrow L, E | E+T | E-T | T^*F | T/F | F\uparrow P | (E) | N(L) | \text{id}$

- 该文法中不存在类似的派生。

# 定理 6-8 CFG化简

- 定理 6-8 对于任意CFG  $G$ ,  $\varepsilon \notin L(G)$ , 存在等价的CFG  $G_1$ ,  $G_1$ 不含无用符号、 $\varepsilon$ -产生式和单一产生式。
- 满足本定理的CFG为化简过的文法。
- 已有去无用符号和去 $\varepsilon$ -产生式的结论, 所以只讨论去单一产生式的问题。
- 构造 $G_2$ :

- (1) 如果 $A \rightarrow \alpha$ 不是单一产生式, 则将 $A \rightarrow \alpha$ 放入 $P_2$ ;
- (2) 如果 $A \Rightarrow^+_G B$ , 且 $B \rightarrow \alpha$ 不是单一产生式, 则将 $A \rightarrow \alpha$ 放入 $P_2$ ;
- (3) 在去除单一产生式过程中, 要考虑经过多步推导出的情况, 例如 $A \Rightarrow B \Rightarrow C$ ,  $C \rightarrow \alpha$ 不是单一产生式, 不要忘记将 $A \rightarrow \alpha$ 也要放入 $P_2$ ;

# 定理 6-8 CFG化简

- 对于一个给定的CFG，可以按照如下过程进行化简：
  - (1) 删除无用符号；
  - (2) 删除 $\varepsilon$ -产生式；
  - (3) 删除单一产生式；
  - (4) 当删除单一产生式后，文法中再出现新的无用符号时，再次进行删除无用符号。
- 将满足定理6-8的CFG称为化简过的CFG。

去掉单一产生式后可能会出现新的无用符号，所以需要再次去除无用符号。

例：

$$S \rightarrow A|B$$
$$A \rightarrow a$$
$$B \rightarrow bB|b$$

结果1：

$$S \rightarrow a|bB|b$$
$$A \rightarrow a$$
$$B \rightarrow bB|b$$

其中A不再出现在任何句型中，应该删掉。

结果2：

$$S \rightarrow a|bB|b$$
$$B \rightarrow bB|b$$

# 例子

---

- 例：对下面文法进行化简：

$S \rightarrow ABCDE | aB | fF | \varepsilon$

$A \rightarrow aBCA | BC | \varepsilon$

$B \rightarrow b | bB | \varepsilon$

$C \rightarrow c | cC | \varepsilon$

$D \rightarrow d | dD | \varepsilon$

$E \rightarrow e | eE | \varepsilon$

$F \rightarrow fF$

$G \rightarrow g | gG$

# 例子

---

## 1、去无用符号

(1) 执行算法6-1，删除派生不出终极符号行的变量，F是无用符号，  
 $NEWV = \{S, A, B, C, D, E\}$

(2) 执行算法6-2，删除不出现在任何句型中的语法符号，G, f, g为无用符号  
 $NEWV = \{S, A, B, C, D, E\}$   
 $NEWT = \{a, b, c, d, e\}$

化简结果：

$$\begin{aligned} S &\rightarrow ABCDE|aB|\varepsilon \\ A &\rightarrow aBCA|BC|\varepsilon \\ B &\rightarrow b|bB|\varepsilon \\ C &\rightarrow c|cC|\varepsilon \\ D &\rightarrow d|dD|\varepsilon \\ E &\rightarrow e|eE|\varepsilon \end{aligned}$$

# 例子

- 2、去空产生式

(1) 求可空变量集

- $U = \{S, A, B, C, D, E\}$

(2) 先考虑对S的产生式进行去空产生式化简：

- $S \rightarrow ABCDE$
- $S \rightarrow BCDE | ACDE | ABDE | ABCE | ABCD$  (一个变量用空替代)
- $S \rightarrow ABC | ABD | ABE | ACD | ACE | ADE | BCD | BCE | BDE | CDE$  (两个变量用空替代)
- $S \rightarrow AB | AC | AD | AE | BC | BD | BE | CD | CE | DE$  (三个变量用空替代)
- $S \rightarrow A | B | C | D | E$  (四个变量用空替代)
- $S \rightarrow aB | a$
- $S \rightarrow \varepsilon$  (开始符号的空产生式单独保留)

(3) 考虑对其他变量产生式进行去空产生式化简：

- $A \rightarrow aBCA | aCA | aBA | aBC | aB | aC | aA | BC | B | C | a$
- $B \rightarrow b | bB \quad C \rightarrow c | cC \quad D \rightarrow d | dD \quad E \rightarrow e | eE$

$$\begin{aligned} S &\rightarrow ABCDE | aB | \varepsilon \\ A &\rightarrow aBCA | BC | \varepsilon \\ B &\rightarrow b | bB | \varepsilon \\ C &\rightarrow c | cC | \varepsilon \\ D &\rightarrow d | dD | \varepsilon \\ E &\rightarrow e | eE | \varepsilon \end{aligned}$$

# 例子

- 3、去单一产生式

- 在2中化简得到的产生式组中， $S \rightarrow A|B|C|D|E$ 、 $A \rightarrow B|C$ 是单一产生式，下面将这种产生式去除。

$A \rightarrow aBCA|aCA|aBA|aBC|aB|aC|aA|BC|B|C|a$   
 $B \rightarrow b|bB \quad C \rightarrow c|cC \quad D \rightarrow d|dD \quad E \rightarrow e|eE$

- 首先考虑去掉 $A \rightarrow B|C$

- $A \rightarrow aBCA|aCA|aBA|aBC|aB|aC|aA|BC|a$
- $A \rightarrow b|bB \quad A \rightarrow c|cC$

- 合并： $A \rightarrow aBCA|aCA|aBA|aBC|aB|aC|aA|BC|a|b|bB|c|cC$

- 再考虑去掉 $S \rightarrow A|B|C|D|E$

- $S \rightarrow aBCA|aCA|aBA|aBC \quad S \rightarrow b|bB \quad S \rightarrow c|cC \quad S \rightarrow d|dD \quad S \rightarrow e|eE$

- 合并：

- $S \rightarrow aBCA|aCA|aBA|aBC$
- $S \rightarrow a|b|c|d|e|aA|aB|aC|bB|cC|dD|eE|BC$

# 例子

---

- 将化简后的产生式合并，最后化简文法为：
  - $S \rightarrow ABCDE | BCDE | ACDE | ABDE | ABCE | ABCD | ABC | ABD | ABE | ACD | ACE | ADE | BCD | BCE | BDE | CDE | AB | AC | AD | AE | BC | BD | BE | CD | CE | DE | a | b | c | d | e | aA | aB | aC | bB | cC | dD | eE | BC | aBCA | aCA | aBA | aBC | \varepsilon$
  - $A \rightarrow aBCA | aCA | aBA | aBC | aB | aC | aA | BC | a | b | bB | c | cC$
  - $B \rightarrow b | bB \quad C \rightarrow c | cC$
  - $D \rightarrow d | dD \quad E \rightarrow e | eE$
- 4、运用算法6-1和算法6-2,发现无“无用符号”,以上文法就是原文法的化简结果

# 回顾

---

- 理解上下文无关文法定义语言的方式。
- 理解派生树概念，理解文法及语言的二义性问题。
- 理解基于CFG进行语言分析的方法。
- 理解为何要对CFG进行化简，掌握文法化简方法，包括去无用符号、去空产生式、去单一产生式等。